30-DAY AI & HYBRID DEEP LEARNING SPRINT • PHASE 1: CLASSICAL ML MECHANICS

Day 01: Gradient Descent Visualizer

Experiencing Iterative Weight Optimization, Loss Surfaces, and Learning Rate Dynamics from Scratch

1. Why Build Gradient Descent Without Libraries?

Core Intuition

Every modern architecture you will build in Phases 3 and 4—from Convolutional Neural Networks (CNNs) to Hybrid Vision Transformers—relies on the exact same engine under the hood: **iterative first-order optimization**. When you call `loss.backward()` and `optimizer.step()` in PyTorch, the framework computes the local slope (gradient) of the loss landscape with respect to every weight and nudges the parameters in the opposite direction. Building this loop in pure NumPy removes the black box so you can physically see why models stall, oscillate, or explode.

2. The 4-Step Mathematical Engine

Forward & Backward Pass

In univariate linear regression, our model attempts to recover a hidden linear law $\mathbf{y = 3.5x + 2.0 + \varepsilon}$ from $\mathbf{N = 100}$ noisy observations by tuning two scalar parameters: slope $\mathbf{w}$ and intercept $\mathbf{b}$.

STEP A • FORWARD HYPOTHESIS & MSE LOSS

$$\hat{y}_i = w \cdot x_i + b \quad | \quad L(w, b) = (1 / N) \sum (\hat{y}_i - y_i)^2$$

Squaring the residual $e_i = (\hat{y}_i - y_i)$ penalizes large outliers heavily and creates a smooth, convex parabolic bowl with a single global minimum.

STEP B • ANALYTICAL GRADIENTS & PARAMETER UPDATE

$$\partial L/\partial w = (2/N) \sum x_i(\hat{y}_i - y_i) \quad | \quad w \leftarrow w - \alpha(\partial L/\partial w)$$

By the Chain Rule, the gradient points uphill toward maximum error. Subtracting $\alpha \cdot \nabla L$ steps downhill toward minimum error; bias uses $\partial L/\partial b = (2/N) \sum (\hat{y}_i - y_i)$.

3. Visualizing Learning Rate Regimes on the Loss Bowl

Hyperparameter Geometry

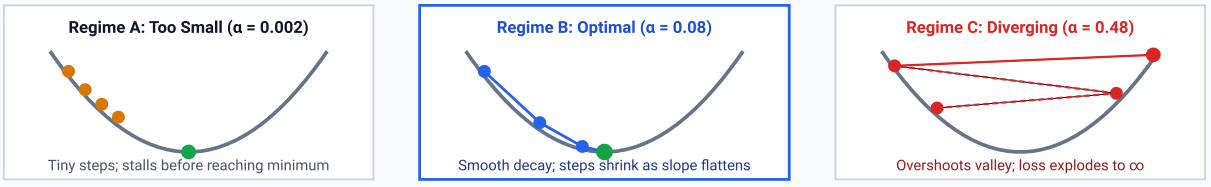


Figure 1.1: How the learning rate scalar α governs trajectory stability across the convex MSE loss surface L(w).

4. Vectorized Tensor Shapes & Implementation Reference

NumPy Architecture

A critical habit for deep learning is tracking array shapes at every line of code and replacing slow Python `for` loops with vectorized matrix operations. The table below maps each mathematical equation to its exact NumPy vector operation and shape.

OPERATION STAGE	MATHEMATICAL EXPRESSION	VECTORIZED NUMPY CODE	ARRAY SHAPE
1. Input & Target	Feature vector $\mathbf{x}$, Target $\mathbf{y}$	`x, y` (1D float64 arrays)	`(100,)` each
2. Forward Pass	$\hat{\mathbf{y}} = \mathbf{w} \cdot \mathbf{x} + \mathbf{b}$	`y_hat = w * x + b`	`(100,)` broadcast
3. Residual Error	$\mathbf{e} = \hat{\mathbf{y}} - \mathbf{y}$	`err = y_hat - y`	`(100,)` vector
4. Loss Scalar	$L = (1/N) \sum e_i^2$	`loss = np.mean(err ** 2)`	`()` scalar float
5. Gradients	$\partial L/\partial w, \partial L/\partial b$	`dw = (2/N) * np.dot(x, err)` `db = 2 * np.mean(err)`	`()` scalar floats

5. Your 90-Minute Daily Execution Protocol

Time-Boxed Workflow

1. **Minutes 00–15 (Generate & Inspect):** Paste the Antigravity prompt into your editor and run `day01_gradient_descent.py`. Verify that the terminal table prints the final learned weights near $\mathbf{w \approx 3.50, b \approx 2.00}$ for the optimal learning rate ($\boldsymbol{\alpha = 0.08}$).
2. **Minutes 15–45 (Break the Hyperparameters):** Open the script and intentionally test edge cases. Set `lr = 0.52` and observe how the loss curve shoots upward on the log plot and the contour trajectory arrows bounce across opposite walls of the valley.
3. **Minutes 45–70 (Feature Scaling Experiment):** Multiply the input feature `x` by `10.0` (changing its range from $\mathbf{[-2, 2]}$ to $\mathbf{[-20, 20]}$) while keeping `lr = 0.08`. Notice how the contour bowl stretches into a narrow canyon and immediately diverges! This proves firsthand why **Batch Normalization** and **StandardScaler** are mandatory in deep neural networks.
4. **Minutes 70–90 (Save Visual Proof):** Restore clean settings, export `day01_gradient_descent_artifact.png`, and commit your script and plot to your repository.

> **How Day 1 Connects to Phase 3 & Phase 4 Hybrid Models**
>
> In Day 1, your parameter space is 2-dimensional ($\mathbf{w, b}$), allowing us to plot the entire contour surface directly. When you build the **CNN + LSTM Video Hybrid (Day 23)** or **Multimodal CNN + MLP Fusion (Day 25)**, the exact same gradient update rule operates simultaneously across millions of dimensions. Understanding how unscaled inputs stretch the loss canyon today explains why multimodal fusion networks fail to converge when image embeddings and tabular features have mismatched variance tomorrow.